

LE STRUCT

Le **struct** in C# sono **tipi valore** (Value Type) e vengono utilizzate per rappresentare oggetti **leggeri e immutabili** che non necessitano dell'overhead della gestione della memoria dei tipi riferimento (Reference Type).

Caratteristiche principali delle struct:

- **Tipo valore (Value Type)** → Memorizzato nello stack.
- **Non ha garbage collection** come gli oggetti di classe.
- **Perfetto per dati semplici** (coordinate, colori, punti, ecc.).
- **Può avere metodi, costruttori e proprietà** (ma senza costruttore vuoto predefinito!).

```
struct Punto
{
    public int X;
    public int Y;

    // Costruttore
    public Punto(int x, int y)
    {
        X = x;
        Y = y;
    }

    // Metodo per spostare il punto
    public void Muovi(int dx, int dy)
    {
        X += dx;
        Y += dy;
    }

    // Metodo per stampare le coordinate
    public void Stampa()
    {
        Console.WriteLine($"Punto: ({X}, {Y})");
    }
}

class Program
{
    static void Main()
    {
        Punto p = new Punto(5, 10); // Creazione di una struct
        p.Stampa();

        p.Muovi(3, -2);
        p.Stampa();
    }
}
```

Passaggio di struct per valore e per riferimento

Di default, una struct viene passata **per valore**.

Per modificarla direttamente in un metodo, **usiamo ref o out**

```
static void ModificaPunto(Punto p)
{
    p.X += 10;
    p.Y += 10;
}

Punto p = new Punto(3, 3);
ModificaPunto(p); // Non cambia nulla perché è passato per valore
Console.WriteLine($"Dopo: ({p.X}, {p.Y})"); // Output: (3, 3)
```

```
static void ModificaPuntoRef(ref Punto p)
{
    p.X += 10;
    p.Y += 10;
}

Punto p = new Punto(3, 3);
ModificaPuntoRef(ref p); // Cambia i valori della struct
Console.WriteLine($"Dopo: ({p.X}, {p.Y})"); // Output: (13, 13)
```

```
static void InizializzaPunto(out Punto p)
{
    p = new Punto(100, 100);
}

Punto p;
InizializzaPunto(out p);
Console.WriteLine($"Dopo: ({p.X}, {p.Y})"); // Output: (100, 100)
```

Differenza tra Struct e Class

Caratteristica	Struct	Class
Tipo	Valore	Riferimento
Memoria	Stack	Heap
Garbage Collection	✗ No	✓ Sì
Ereditarietà	✗ No	✓ Sì
Costruttore vuoto	✗ No (sempre inizializzato)	✓ Sì
Usi consigliati	Dati semplici, performance	Oggetti complessi, ereditarietà